

FLOOD-X

SIH Project Implementation Blueprint

AI + Physics Based Urban Flood Intelligence & Emergency Decision Support

1. Project Goal

Build a working prototype that predicts street-level urban flood risk, explains the main causes, allows authorities to test what-if scenarios, and generates flood-aware emergency routes.

2. Core MVP Flow

RAIN → TERRAIN → DRAINAGE → FLOOD PREDICTION → EXPLANATION → WHAT-IF SIMULATION → SAFE ROUTE → ACTION

3. What We Will Demonstrate

- A selected 5–20 km² pilot urban area on an interactive GIS map.
- Rainfall input and a NOW / +30 / +60 / +90 minute prediction timeline.
- Dynamic flood-risk zones at road/grid level.
- Flood probability, estimated depth, and time-to-flood for a selected location.
- An explanation panel showing contributing factors such as rainfall, elevation, runoff/flow accumulation, and drainage overload.
- What-if simulation for increased rainfall and drainage blockage.
- Emergency routing that penalizes roads with high predicted flood risk.

4. Recommended Technology Stack

Layer	Technology	Purpose
Frontend	React + Vite + Tailwind CSS	Dashboard and controls
GIS Map	MapLibre GL JS or Leaflet	Map, layers, flood zones, roads
Backend	Python + FastAPI	APIs and prediction services
ML	scikit-learn + XGBoost	Flood-risk prediction
Geospatial	GeoPandas + Rasterio + Shapely	Vector/raster processing
Database	PostgreSQL + PostGIS	Spatial data and predictions
Routing	A* / Dijkstra	Flood-aware route calculation
Data format	GeoJSON + GeoTIFF + CSV/JSON	Maps, terrain and sensor data

5. System Architecture

Rainfall Data → Data Processing → Terrain/Drainage/Land-use Features → AI + Physics Flood Model → Risk Map + Explanation → What-if Engine → Flood-aware Routing → React GIS Dashboard

6. Module-by-Module Implementation

Module 1 — GIS Dashboard

Create the main map. Add base map, roads, drainage, flood-risk polygons/grid, selected location, timeline controls and emergency route. Keep the UI focused on decision-making rather than showing every dataset.

Module 2 — Rainfall Service

Create an API that returns current/historical rainfall and forecast/synthetic rainfall for the pilot area. For the first prototype, a CSV/JSON time series is acceptable. Later replace it with live data.

Module 3 — Terrain Processing

Use a DEM to derive elevation, slope, flow direction and flow accumulation. Convert these into features for each grid cell or road segment.

Module 4 — Drainage Model

Represent drains/pipes as a network of nodes and edges. Store capacity, current flow and blockage. Calculate effective capacity and utilization. Mark overloaded segments.

Module 5 — Flood Prediction

Start with a transparent baseline risk score using rainfall, elevation, slope, flow accumulation and drainage. Then train Random Forest or XGBoost using historical flood labels when sufficient data is available.

Module 6 — Explainability

For every prediction, return the main contributing features. Present them as percentages/bars and plain-language reasons.

Module 7 — What-if Simulation

Allow the user to change rainfall and drainage blockage. Recalculate risk and redraw the flood-risk layer.

Module 8 — Emergency Routing

Build a road graph. Use A* or Dijkstra with an edge cost that combines travel distance and flood risk. Compare normal shortest route with the safer FLOOD-X route.

7. Initial Data Model

- Rainfall: timestamp, latitude, longitude, rainfall_mm, intensity.
- Terrain/grid: cell_id, elevation, slope, flow_accumulation.
- Drainage: node_id/edge_id, capacity, current_flow, blockage, utilization.
- Road: road_id, geometry, length, road_type, flood_risk.
- Prediction: location_id, timestamp, probability, depth_cm, time_to_flood_min, risk_level.
- Historical flood: location, timestamp, observed flood/no-flood, observed depth where available.

8. Baseline Flood-Risk Model

For the first working version, normalize each feature to 0–1 and calculate a weighted risk score. Example weights (prototype only):

- Rainfall: 0.30
- Elevation: 0.20
- Slope: 0.15
- Flow accumulation: 0.15
- Drainage condition/utilization: 0.20

Risk levels: 0.0–0.3 Low; 0.3–0.6 Medium; 0.6–0.8 High; 0.8–1.0 Critical. These are prototype thresholds and must be calibrated/validated before real-world deployment.

9. AI/ML Upgrade

Once the baseline pipeline works, create training samples from historical rainfall, terrain, drainage and observed flood events. Train Random Forest/XGBoost to estimate P(flood). Compare the ML model with the baseline. Use feature importance/SHAP-style explanations where appropriate.

10. Flood-Aware Routing

For each road edge, calculate a routing cost such as: $\text{cost} = \text{distance} \times (1 + \text{flood_risk} \times \text{penalty})$. High-risk roads therefore become expensive to traverse. Run A* or Dijkstra and display both routes.

11. API Plan

Endpoint	Method	Purpose
/api/rainfall/current	GET	Current rainfall
/api/flood/risk-map	GET	Flood-risk GeoJSON/grid
/api/flood/prediction/{id}	GET	Prediction for a location
/api/flood/explain/{id}	GET	Contributing factors
/api/simulation	POST	Run rainfall/blockage scenario
/api/route	POST	Generate flood-aware route

12. Suggested Project Structure

```
flood-x/  
├── frontend/  
│   └── src/  
│       ├── components/  
│       ├── pages/  
│       ├── map/  
│       └── api/  
├── backend/  
│   └── app/  
│       ├── main.py  
│       └── api/
```

```
|   ├── services/
|   ├── models/
|   └── utils/
|── ml/
|   ├── training/
|   ├── models/
|   └── notebooks/
|── data/
|   ├── rainfall/
|   ├── elevation/
|   ├── drainage/
|   ├── roads/
|   └── historical/
└── README.md
```

13. 10-Step Development Plan

1. Choose one pilot urban zone and define its boundary.
2. Build the React GIS dashboard and display roads/base layers.
3. Prepare rainfall, DEM, drainage and road data.
4. Build FastAPI and connect the frontend.
5. Implement the baseline flood-risk calculation.
6. Display dynamic risk and NOW/+30/+60/+90 minute views.
7. Add explanation for each high-risk location.
8. Add rainfall and drainage-blockage what-if simulation.
9. Add flood-aware A*/Dijkstra routing.
10. Add ML, validation, alerts and final UI polish.

14. SIH Demo Scenario

1. Start with a normal map.
2. Introduce heavy rainfall.
3. Move the timeline to +60 minutes.
4. Show one road becoming Critical with probability, depth and time-to-flood.
5. Click WHY and show rainfall + low elevation + drainage overload.
6. Set drainage blockage to 40% and run WHAT-IF.

7. Show the affected flood area increasing.
8. Select an ambulance and destination.
9. Show the normal shortest route and the FLOOD-X lower-risk route.
10. Finish with: “We don't just predict where water will go. We predict where people can safely go.”

15. Team Division

Role	Main Responsibility	Deliverable
Frontend/GIS	React dashboard, map, timeline, charts	Working UI
Backend	FastAPI, APIs, database	Working services
Data/Geospatial	DEM, roads, drainage, rainfall processing	Feature layers
AI/ML	Risk model, training, explanation	Prediction engine
Routing/Simulation	What-if model and safe routing	Scenario + route engine
Integration/Presentation	Deployment, testing, SIH demo	End-to-end demo

16. Validation and Safety

Treat the SIH system as a prototype decision-support tool, not an operational emergency authority. Validate predictions against observed events before claiming accuracy. Clearly label simulated/demo data. For real deployment, integrate authoritative rainfall, drainage, road-closure and emergency-service data, and establish human approval workflows.

17. Definition of Done for SIH

- The map loads the selected pilot area.
- Rainfall input changes the predicted risk.
- Risk is visible at street/grid level.
- Clicking a location shows probability, depth and time-to-flood.
- The system explains the major risk factors.
- Changing drainage blockage changes the predicted risk.
- The emergency route avoids high-risk road segments.
- The complete flow works live without manually editing database records during the demo.

18. Key Principle

Do not try to build the entire city. Build one small area extremely well.

The SIH prototype should prove one complete loop: rainfall → prediction → explanation → simulation → safe route. That is more convincing than showing many unfinished features.

19. Real APIs and Datasets

FLOOD-X should use a combination of India-specific live weather data, open geospatial datasets, satellite rainfall, terrain data, road networks, population exposure data, and a project-specific drainage dataset. The following sources are recommended for the SIH prototype.

19.1 Recommended Data Sources

Data	Recommended Source	Use in FLOOD-X	Priority	Format
Real-time / forecast rainfall	India Meteorological Department (IMD) APIs	Current rainfall, district/station nowcast, warnings and precipitation inputs	Must Have	JSON/API
Historical precipitation	NASA GPM IMERG	ML training, historical rainfall patterns and gap filling	Must Have	NetCDF/HDF/GeoTIFF-compatible workflows
Elevation / terrain	Copernicus DEM GLO-30 / GLO-90	Elevation, slope, flow direction, flow accumulation and low-lying areas	Must Have	GeoTIFF
Road network / POIs	OpenStreetMap + Overpass API	Road graph, hospitals, fire stations, police stations and routing	Must Have	OSM/GeoJSON
Flood hazard / thematic layers	ISRO Bhuvan	Supporting flood-hazard context and Indian geospatial layers	Recommended	GIS layers
Population exposure	WorldPop	Estimate population exposed to predicted flood zones	Recommended	Raster/API
Drainage network	Municipal/open data + custom pilot dataset	Drain capacity, utilization and blockage what-if simulation	Must Have	CSV/GeoJSON/PostGIS
Historical flood observations	Municipal records / field observations / project labels	Ground-truth labels for model validation and ML training	Must Have	CSV/GeoJSON/PostGIS

19.2 IMD API Integration

IMD should be the primary India-specific rainfall/nowcast source for the live prototype. Useful documented API categories include district rainfall, district nowcast, station/AWS observations, warnings and basin QPF. Keep the exact endpoint configurable because API paths and access requirements can change.

Example API categories:
GET /district rainfall
GET /district nowcast

```
GET /station/AWS observations
GET /district warnings
GET /basin QPF
```

FLOOD-X flow:

```
IMD API -> rainfall normalization -> spatial interpolation/grid -> flood model -> risk map
```

- Store API credentials and endpoint URLs in environment variables; never hard-code secrets.
- Cache recent observations so the dashboard remains usable if the upstream service is temporarily unavailable.
- Normalize all rainfall values to a common unit, timestamp and spatial grid before model inference.

19.3 NASA GPM IMERG

Use NASA GPM IMERG as the historical precipitation source for model development. It is particularly useful for building time-aligned rainfall features such as 30-minute, 1-hour, 3-hour, 6-hour and 24-hour accumulation.

- Create rainfall features: rain_30m, rain_1h, rain_3h, rain_6h, rain_24h.
- Join rainfall grids with flood observation locations using spatial and temporal matching.
- Use historical rainfall for training/validation rather than treating current API observations as training labels.

19.4 Copernicus DEM

Use a Copernicus Digital Elevation Model for the pilot zone. Process the DEM into terrain features that directly support the flood model.

- Elevation
- Slope
- Flow direction
- Flow accumulation
- Depression / low-point detection
- Optional derived catchment boundaries

Recommended processing chain: **DEM -> reprojection -> clipping -> hydrological conditioning -> slope/flow accumulation -> feature rasters**

19.5 OpenStreetMap + Overpass API

OpenStreetMap provides the road graph and useful emergency-service points. Overpass can query selected OSM objects for the pilot area.

Example Overpass query pattern:

```
[out:json];
way["highway"] (SOUTH, WEST, NORTH, EAST);
out geom;
```

- Convert road features into a graph of nodes and edges.
- Attach flood probability, predicted depth and road-passability penalty to each road edge.
- Use hospitals, fire stations and police stations as emergency-routing origins/destinations.

19.6 ISRO Bhuvan

Bhuvan can provide supporting Indian geospatial and thematic context, including flood-hazard and land-use-related layers where appropriate. Treat such layers as supporting evidence and not automatically as ground truth.

- Use flood-hazard information as a prior/susceptibility feature.
- Use land-use/land-cover information to improve runoff estimation.
- Document the source, date and resolution of every imported layer.

19.7 WorldPop Population Exposure

Population raster/API data can be combined with FLOOD-X predicted flood polygons to estimate exposed population.

Exposure calculation:

```
exposed_population = sum(population_cells intersecting predicted_flood_zone)
```

Priority score (prototype):

```
priority = flood_risk * exposed_population_factor
```

19.8 Drainage Dataset

Because detailed municipal drainage networks are often unavailable as a clean public API, create a structured pilot drainage dataset and replace/augment it with verified municipal data when available.

drain_id	type	lat	lon	capacity_m3s	blockage_percent
D001	culvert	10.012	76.302	2.5	10
D002	storm_drain	10.015	76.305	1.8	20
D003	culvert	10.018	76.309	3.2	0

- Represent drains as nodes and links where possible.
- Calculate effective capacity after blockage: $\text{effective_capacity} = \text{nominal_capacity} * (1 - \text{blockage_percent}/100)$.
- Use blockage as a what-if simulation parameter, not as a measured fact unless supported by field data.

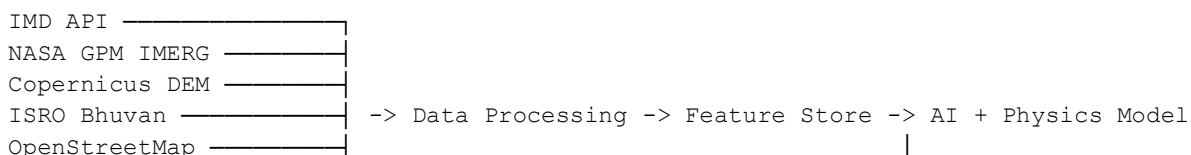
19.9 Historical Flood Dataset

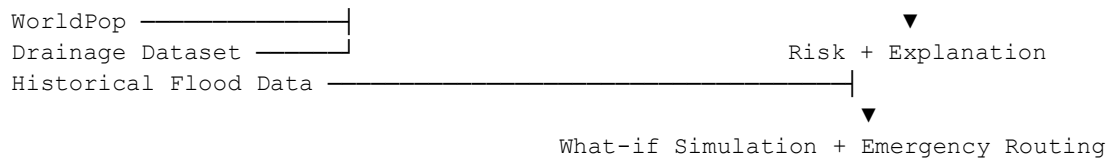
Build a project-specific labelled dataset by joining historical rainfall, terrain, drainage, road and observed flood information.

```
timestamp,lat,lon,rain_mm_1h,rain_mm_3h,elevation,slope,flow_accumulation,  
drain_capacity,land_use,flood_depth_cm,flood_observed
```

20. Data Integration Architecture

The following data flow should be implemented in the backend. Raw external data is normalized first, then converted into a common spatial grid/road graph before entering the prediction and routing layers.





21. API Layer for FLOOD-X Backend

Endpoint	Method	Purpose	Main Data
/api/rainfall/current	GET	Current rainfall observations	IMD
/api/rainfall/forecast	GET	Short-horizon rainfall/nowcast	IMD
/api/flood/risk-map	GET	Current/forecast flood-risk grid	Model
/api/flood/prediction/{id}	GET	Risk, depth and time-to-flood for a location	Model
/api/flood/explain/{id}	GET	Top factors contributing to risk	Model/feature data
/api/simulation	POST	Rainfall/drainage what-if simulation	Simulation engine
/api/route	POST	Flood-aware emergency route	OSM + flood risk
/api/exposure	GET	Population exposure estimate	WorldPop + risk
/api/health	GET	Service health and data-source status	Backend

22. Suggested Environment Variables

```

IMD_BASE_URL=...
IMD_API_KEY=...
DATABASE_URL=postgresql://.../floodx
OSM_OVERPASS_URL=...
COPERNICUS_DATA_URL=...
WORLDPOP_DATA_URL=...
MODEL_PATH=./ml/models/flood_model.joblib

```

23. Data Storage Design

Table	Important Fields	Source
rainfall_observations	timestamp, lat, lon, rainfall_mm, source	IMD / IMERG
terrain_cells	cell_id, elevation, slope, flow_accumulation	Copernicus DEM
drainage_nodes	node_id, geometry, capacity, blockage	Municipal/custom
drainage_edges	edge_id, from_node, to_node, capacity	Municipal/custom
roads	road_id, geometry, length, road_type	OpenStreetMap

flood_predictions	timestamp, location_id, probability, depth_cm, time_to_flood	FLOOD-X
flood_observations	timestamp, location, observed_depth, flood_observed	Historical/field
population_cells	geometry, population	WorldPop

24. SIH Implementation Priority

1. Phase 1 — IMD rainfall/nowcast + GIS dashboard.
2. Phase 2 — Copernicus DEM + terrain feature generation.
3. Phase 3 — Baseline flood-risk model and forecast timeline.
4. Phase 4 — OpenStreetMap road graph + flood-aware routing.
5. Phase 5 — Drainage dataset + blockage what-if simulation.
6. Phase 6 — Explanation layer and feature contributions.
7. Phase 7 — Historical rainfall/flood dataset + ML model.
8. Phase 8 — WorldPop exposure and final SIH demo polish.

25. Data Quality, Licensing and Validation

- Record source, acquisition date, spatial resolution and processing steps for every dataset.
- Check the terms of use and attribution requirements of each API/dataset before deployment.
- Do not treat simulated drainage blockage or synthetic flood labels as real observations.
- Validate predictions against independent historical/field observations before making operational claims.
- Keep external APIs behind backend adapters so a provider can be replaced without rewriting the ML or frontend layers.
- For the SIH prototype, clearly label demonstration/simulated values where live authoritative data is unavailable.

26. Official Data Source Links

- **IMD API Management Portal:** <https://api.imd.gov.in/public/index.php>
- **NASA Earthdata / GPM:** <https://www.earthdata.nasa.gov/>
- **Copernicus DEM / Copernicus Data Space:** <https://dataspace.copernicus.eu/explore-data/data-collections/copernicus-contributing-missions/collections-description/COP-DEM>
- **OpenStreetMap:** <https://www.openstreetmap.org/>
- **Overpass API:** https://wiki.openstreetmap.org/wiki/Overpass_API
- **ISRO Bhuvan:** <https://bhuvan.nrsc.gov.in/>
- **WorldPop:** <https://www.worldpop.org/>

27. User Location & Custom Data Upload

FLOOD-X should allow a user, citizen, field officer, municipal engineer, or emergency operator to select their location and upload local data. This makes the platform adaptable to locations where official datasets are incomplete or unavailable.

27.1 User Location Selection

- Use browser/mobile GPS permission to obtain the user's current latitude and longitude when available.
- Allow manual location search by city, address, landmark, PIN code, or map click.
- Allow the user to draw/select an area of interest on the map.
- Store only the minimum location information required for the analysis and clearly show when location permission is being used.
- After location selection, automatically load available rainfall, terrain, road and hazard layers for that area.

Example user flow: Allow Location → Map Centers on User → Select Area → Load Data → Run FLOOD-X Analysis → Show Risk + Explanation + Safe Route.

27.2 User-Uploadable Data

Upload Type	Accepted Format	Example	FLOOD-X Use
Rainfall observations	CSV / Excel / JSON	timestamp, lat, lon, rainfall_mm	Local rainfall correction and model input
Flood reports	CSV / GeoJSON / Form	location, time, depth, photo	Ground-truth/validation and live incident layer
Drainage network	GeoJSON / Shapefile / CSV	drain location, capacity, blockage	Drainage-aware prediction and what-if simulation
Elevation / terrain	GeoTIFF / DEM	local DEM	Slope, flow and low-point analysis
Road network	GeoJSON / Shapefile	road geometry, road type	Emergency routing
Land-use / land-cover	GeoJSON / GeoTIFF	built-up, vegetation, water	Runoff and flood-risk features
Historical flood zones	GeoJSON / KML / Shapefile	past inundation polygon	Model calibration and context
Emergency assets	CSV / GeoJSON	hospital, fire station, shelter	Emergency route planning

27.3 Citizen Flood Report

A simple reporting form can turn FLOOD-X into a continuously improving community-data platform.

- Current location: GPS or map pin
- Water depth: none / ankle / knee / waist / numeric centimetres
- Road status: open / partially blocked / closed
- Drain status: normal / overflowing / blocked / unknown
- Optional photograph or short video
- Time of observation
- Optional description

Example: A resident reports 45 cm water depth on a road. FLOOD-X places the report on the map, compares it with the predicted flood layer, and can use verified reports for validation.

27.4 Upload Processing Pipeline

User uploads file / submits report
↓
File type + size + schema validation
↓
Coordinate-system detection / normalization
↓
Geometry and value validation
↓
Convert to common internal format
↓
Store in PostgreSQL/PostGIS or object storage
↓
Generate/update FLOOD-X features
↓
Recalculate risk / exposure / routing

27.5 Upload APIs

Endpoint	Method	Purpose	Input
/api/location/current	GET	Return/validate current user location	GPS permission
/api/location/search	GET	Search/select a location	query
/api/upload/data	POST	Upload CSV/GeoJSON/GeoTIFF/custom dataset	multipart file + metadata
/api/upload/validate/{id}	GET	Show validation errors/warnings	upload_id
/api/upload/preview/{id}	GET	Preview uploaded features/data	upload_id
/api/upload/apply/{id}	POST	Approve and add valid data to project	upload_id
/api/flood/report	POST	Submit citizen/field flood report	location + observation
/api/flood/reports	GET	Retrieve verified/recent reports	area/time filters
/api/user/layers	GET	List user/project-uploaded layers	user/project
/api/user/layers/{id}	DELETE	Remove an uploaded layer	layer_id

27.6 Example Upload Metadata

POST /api/upload/data

Metadata:

```
{
  "dataset_name": "Local Drainage Survey",
  "data_type": "drainage",
  "location": {"lat": 10.012, "lon": 76.302},
  "coordinate_system": "EPSG:4326",
  "source": "Municipal field survey",
  "observed_at": "2026-09-12T10:30:00",
  "use_for_prediction": true
}
```

27.7 Data Validation Rules

- Reject unsupported file types and oversized uploads.
- Check required columns before importing CSV/Excel files.

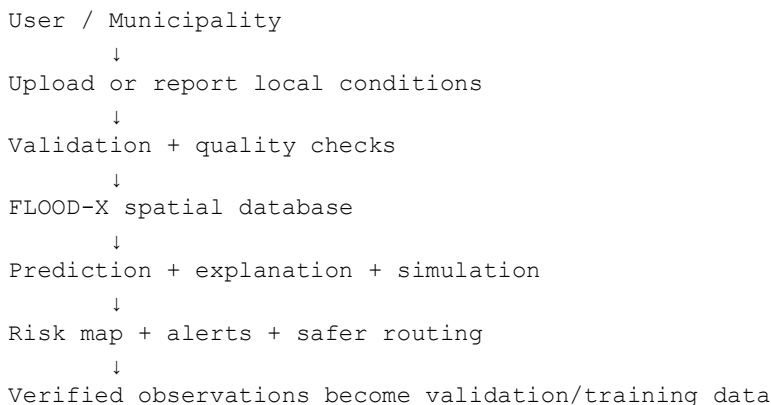
- Validate latitude/longitude ranges and geometry validity.
- Detect missing values, duplicate records and impossible measurements.
- Reproject spatial data into the FLOOD-X project coordinate system.
- Flag data with unknown or suspicious timestamps.
- Keep uploaded data separate from authoritative data until reviewed.
- Allow an administrator/authorized operator to mark a dataset as verified.

27.8 User Dashboard Features

- 📍 Use My Location — centers the map on the user.
- ⬆ Upload Data — upload local GIS/weather/drainage/flood files.
- 🗺️ My Layers — turn uploaded layers on/off.
- 🌧 Report Flood — submit a current flood observation.
- 🔍 Analyze This Area — run FLOOD-X for the selected location.
- ⚖ Compare Prediction vs Report — compare model output with local observations.
- 🚒 Find Safe Route — route emergency vehicles around predicted/observed flooding.
- 🛠 What-if — test rainfall increase or drainage blockage.

27.9 How User Data Improves FLOOD-X

The upload feature should not simply display files. Uploaded and verified observations should feed the decision-support loop.



27.10 Important SIH Differentiator

This creates a two-way system: FLOOD-X consumes authoritative and open datasets, while users and field teams can contribute local observations that improve situational awareness. The platform therefore becomes **location-aware, data-adaptive and community/authority-assisted** rather than a fixed flood map.

28. Expanded End-to-End User Scenario

9. User opens FLOOD-X and selects 'Use My Location' or manually selects a location.
10. FLOOD-X loads available rainfall, terrain, flood-hazard, roads and population layers.
11. User uploads a local drainage CSV or GeoJSON supplied by a municipality/field team.
12. System validates the file and previews the data before applying it.
13. User runs a flood analysis for the selected area.
14. FLOOD-X predicts risk, depth and time-to-flood and explains the major contributing factors.

15. User can submit a current flood report with location, water depth and optional photo.
16. The report appears as a separate observation layer and can be compared with the model.
17. User can simulate increased rainfall or drainage blockage.
18. Emergency personnel can request a route that avoids predicted and verified flooded roads.